



UNIVERSITY OF TABUK

College of Computers and Information Technology

Voice and Image Deepfake Detection System

Prepared by:

Ghida Abdualrhman Alshammri

Student ID: 452009887

Supervised by:

Prof. Abdualaziz Almehmadi

May 2025

Introduction

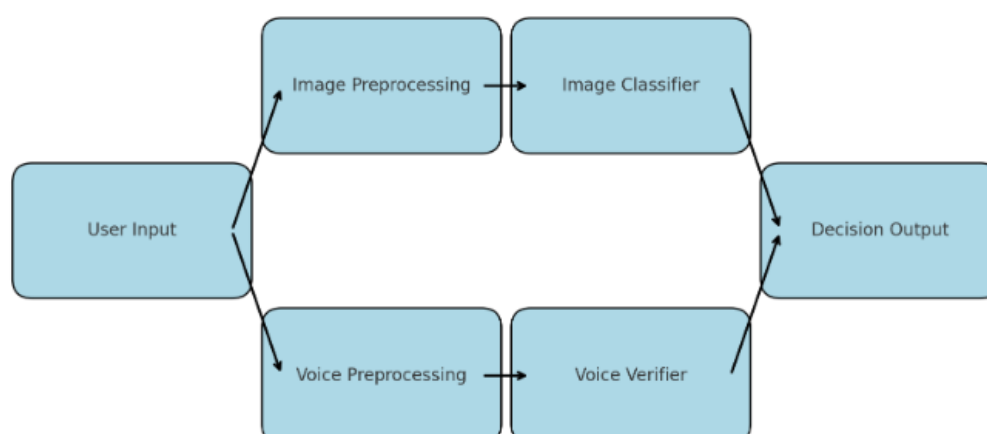
With the increasing spread of AI-generated content, the ability to distinguish between real and fake media has become essential. This project focuses on creating a simple and effective system that can detect whether a given image or voice sample is real or deepfake. The aim is to provide an accessible tool for users to verify the authenticity of multimedia inputs in a clear and user-friendly interface.

Objective

To build a professional GUI-based system that allows users to:

- Upload an image and determine whether it is real or AI-generated.
- Upload a voice sample and determine whether it belongs to the same speaker as a reference or if it's a spoofed (fake) audio.

System Architecture Diagram



Steps Taken

1. Project environment was set up using Python 3.13 and a virtual environment.
2. Required libraries were installed:
 - `speechbrain`, `torchaudio`, `librosa`, `tkinter`, `Pillow`
3. A CNN-based image classification model was implemented and trained.
4. Real and fake facial images were prepared under:

- **data/real** – for authentic images.
 - **data/fake** – for AI-manipulated images.
5. Initial training produced low accuracy due to insufficient data.
 6. More images were added to enhance model generalization.
 7. Model successfully detected real images.
 8. Then tested with an AI-modified image, which was correctly labeled as fake.
 9. A pre-trained **ECAPA-TDNN** model from **speechbrain** was used for voice verification.
 10. Audio files were compared to a reference voice to determine similarity.
 11. Spoofed audio was created using Audacity and tested against the real sample.
 12. A GUI was developed using **Tkinter** to unify the interface for image and voice input.

Image Model Training

To detect image-based deepfakes, a Convolutional Neural Network (CNN) was trained to classify facial images as either real or AI-generated. The dataset used for training included both authentic human faces and artificially modified images created using generative AI tools. These fake samples exhibited a variety of alterations such as facial distortion, lighting inconsistencies, changes in resolution, and synthetic backgrounds. This diversity helped improve the model's ability to generalize and accurately detect deepfakes.

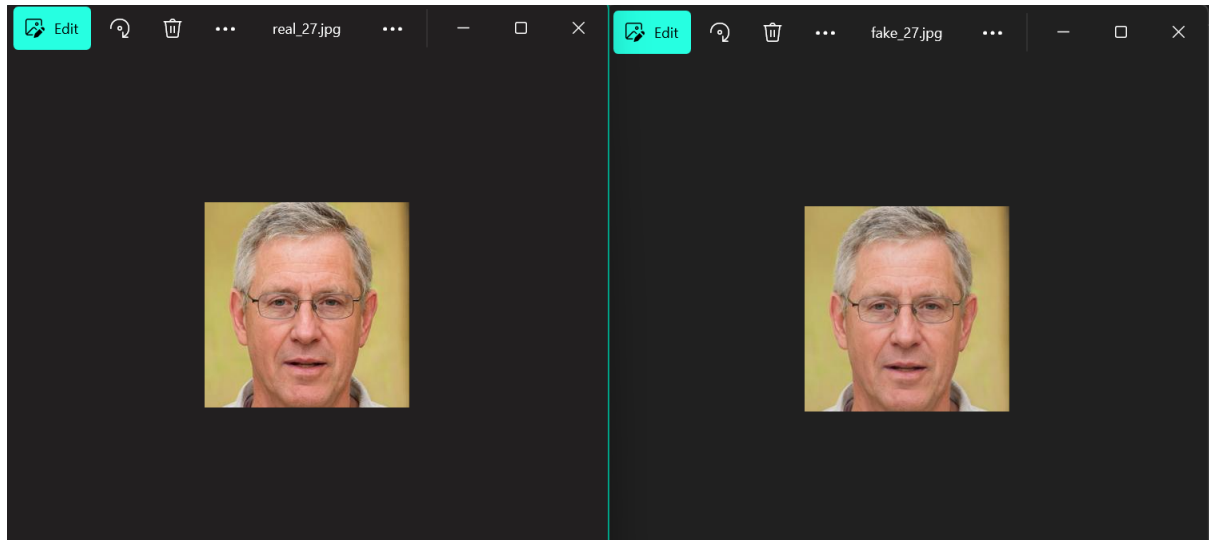


Figure 1: Example of real facial image (left) versus an AI-generated fake version (right).

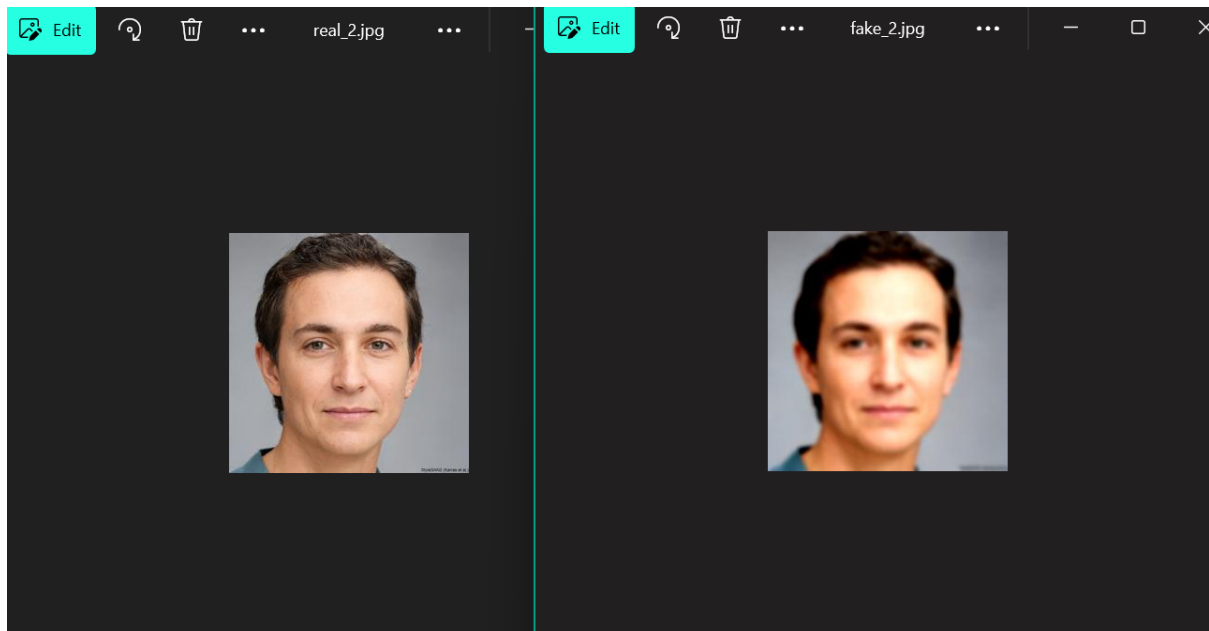


Figure 2: Example of real facial image (left) versus an AI-generated fake version (right).

The dataset was organized into two clearly separated directories:

- `data/real` – containing authentic (real) facial images.
- `data/fake` – containing AI-generated (fake) facial images.

This structured organization enabled the CNN to learn and distinguish between the subtle differences of real and manipulated content effectively.

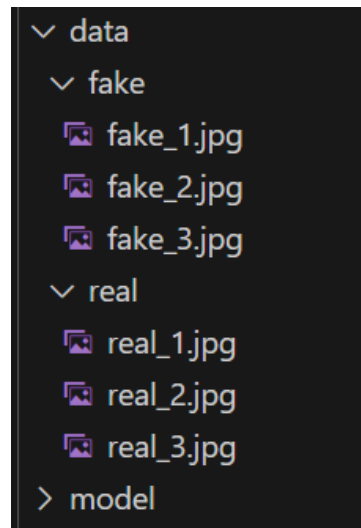


Figure 3: Basic CNN architecture used in training

Initial evaluation of the model yielded low accuracy, mainly due to the limited number of training samples.

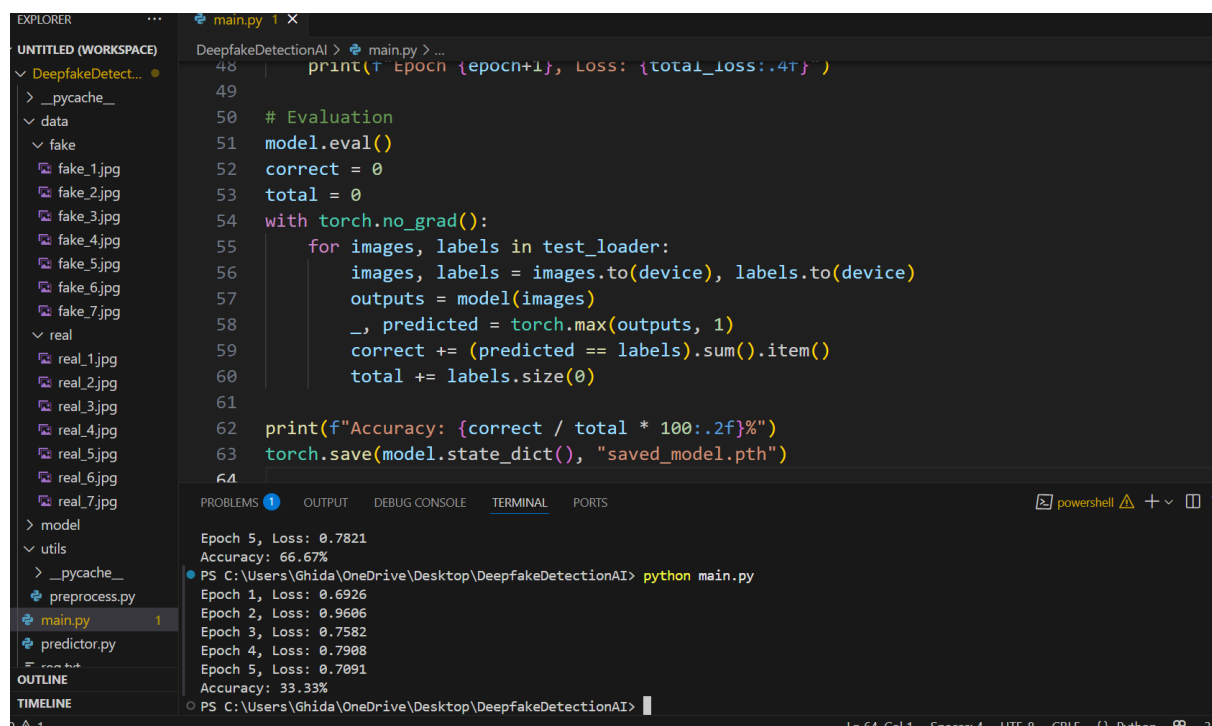


Figure 4: Initial performance evaluation showing low accuracy

To improve model performance, additional data samples were added to increase training diversity and robustness.

```

48 print(f"Epoch {epoch+1}, Loss: {total_loss:.4f}")
49
50 # Evaluation
51 model.eval()
52 correct = 0
53 total = 0
54 with torch.no_grad():
55     for images, labels in test_loader:
56         images, labels = images.to(device), labels.to(device)
57         outputs = model(images)
58         _, predicted = torch.max(outputs, 1)
59         correct += (predicted == labels).sum().item()
60         total += labels.size(0)
61
62 print(f"Accuracy: {correct / total * 100:.2f}%")
63 torch.save(model.state_dict(), "saved_model.pth")
64

```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\Users\Ghida\OneDrive\Desktop\DeepfakeDetectionAI> & C:/Users/Ghida/AppData/Local/Programs/Python/Python313/
Ghida/OneDrive/Desktop/DeepfakeDetectionAI/main.py
Epoch 1, Loss: 0.6936
Epoch 2, Loss: 1.5649
Epoch 3, Loss: 0.6751
Epoch 4, Loss: 1.0001
Epoch 5, Loss: 0.9151
Accuracy: 83.33%

```

Figure 5: Expanded dataset used for retraining the CNN model

Upon retraining with the enhanced dataset, the model was evaluated again using a real image. The model successfully classified the image as genuine.

EXPLORER

- DeepfakeDetect...
- data
- model
- utils
- __pycache__
- preprocess.py
- fake_2.jpg
- main.py
- predict.py
- real_for_prediction.jpg
- req.txt
- saved_model.pth

main.py 1 predict.py real_for_prediction.jpg

DeepfakeDetectionAI > real_for_prediction.jpg

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\Users\Ghida\OneDrive\Desktop\DeepfakeDetectionAI> python predict.py real_for_prediction.jpg
Prediction result: Real ✓
PS C:\Users\Ghida\OneDrive\Desktop\DeepfakeDetectionAI>

```

Figure 6: Prediction result for a real image (classified as real)

Finally, the model was tested with an AI-modified image. It accurately detected the image as fake, confirming the effectiveness of the approach.

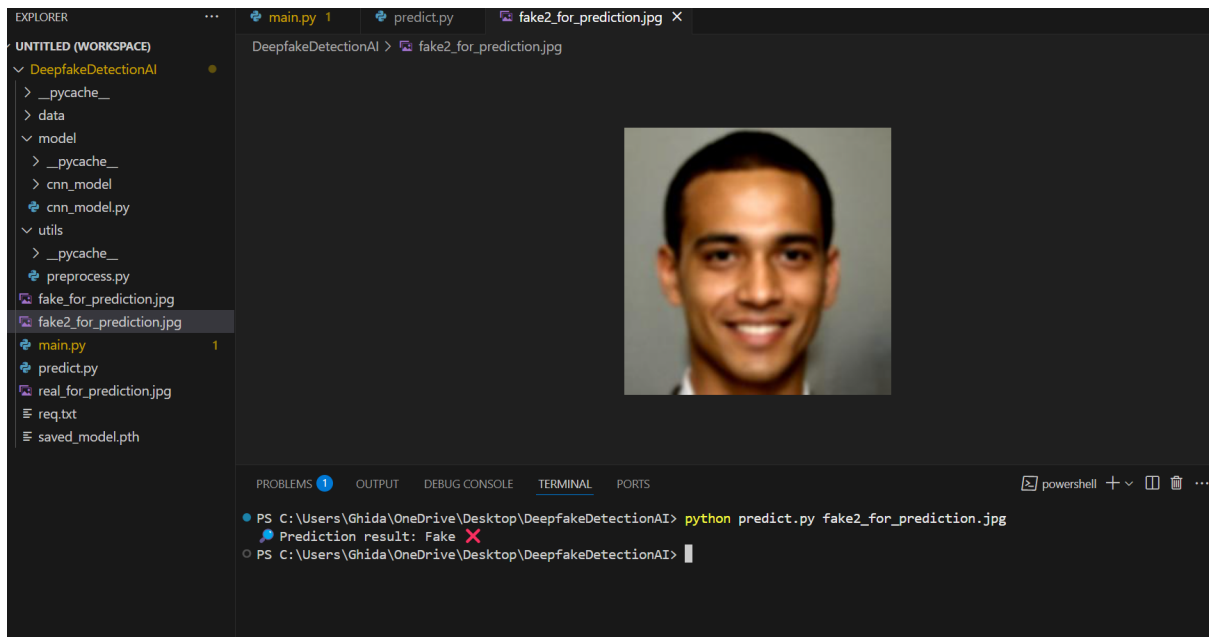


Figure 7: Prediction result for a fake image (classified as fake)

Voice Model Testing

The second major component of this project involved detecting whether a voice sample was authentic or AI-generated. The primary objective was to determine if the input voice belongs to the original speaker or if it has been artificially altered or spoofed.

To achieve this, I utilized the **speechbrain** library alongside the pre-trained ECAPA-TDNN model, specifically designed for speaker verification. The system compares two audio files: a known real voice (reference) and a test voice input. Based on this comparison, it produces a similarity score and a prediction that classifies the test voice as either genuine or fake.

Users are required to upload only one test voice sample. The system automatically compares it against a stored reference voice sample. A high similarity score indicates the voice is real, while a low score suggests it has likely been spoofed.

```

DeepfakeDetectionAI > voice_spoof_detector > voice_spoof_detector.py > ...
1  import librosa
2  import numpy as np
3  import sys
4  import os
5  from scipy.spatial.distance import cosine
6
7  def extract_embedding(path):
8      y, sr = librosa.load(path, sr=16000)
9      mfcc = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=13)
10     embedding = np.mean(mfcc, axis=1)
11     return embedding
12
13 def is_fake(real_path, test_path, threshold=0.3):
14     real_embedding = extract_embedding(real_path)
15     test_embedding = extract_embedding(test_path)
16     similarity = 1 - cosine(real_embedding, test_embedding)
17
18     print(f"\n💬 Similarity Score: {similarity:.4f}")
19     if similarity >= threshold:
20         print("✅ Same Speaker (or very close)")
21     else:
22         print("❌ Likely Fake or Different Speaker")
23
24 if __name__ == "__main__":

```

Figure 8: Voice sample upload interface

```

22     print("❌ Likely Fake or Different Speaker")
23
24 if __name__ == "__main__":
25     if len(sys.argv) not in [3, 4]:
26         print("Usage: python voice_spoof_detector.py real.wav test.wav [threshold]")
27     else:
28         real = sys.argv[1]
29         test = sys.argv[2]
30         threshold = float(sys.argv[3]) if len(sys.argv) == 4 else 0.3
31
32         if not os.path.exists(real) or not os.path.exists(test):
33             print("❌ One or both audio files do not exist.")
34         else:
35             is_fake(real, test, threshold)
36

```

Figure 9: Voice processing and analysis interface

The test audio samples were sourced from freesound.org, a platform offering open-access sound recordings. One of the samples was edited using Audacity—an open-source audio editor—to simulate a spoofed voice, either modified via AI techniques or designed to imitate another speaker. This edited sample served as the fake input.

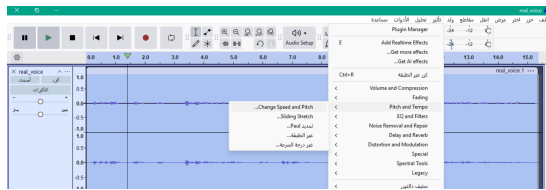


Figure 10: Real voice waveform visualization

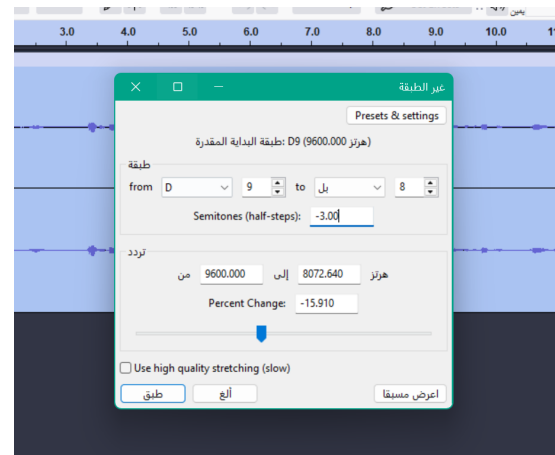


Figure 11: Fake (AI-modified) voice waveform visualization

After configuring the voice model, I conducted a test by uploading a real voice and comparing it against the modified version. The system effectively computed the similarity score and correctly identified the authenticity of the sample.

```

12 def is_fake(real_path, test_path, threshold=0.3):
13     ...
14     ...
15     ...
16     ...
17     ...
18     ...
19     ...
20     else:
21         print("✗ Likely Fake or Different Speaker")
22
23 if __name__ == "__main__":
24     if len(sys.argv) != 3:
25         print("Usage: python simple_voice_check.py real.wav test.wav")
26     else:
27         real = sys.argv[1]
28         test = sys.argv[2]
29         is_fake(real, test)

```

```

PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL PORTS
atformatdirs-4.3.7 pooch-1.8.2 soxr-0.5.0.post1 standard-aifc-3.13.0 standard-chunk-3.13.0 standard-sunau-3.13.0
[notice] A new release of pip is available: 25.0 -> 25.1
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Users\Ghida\OneDrive\Desktop\DeepfakeDetectionAI> C:\Users\Ghida\AppData\Local\Programs\Python\Python313\python.exe voice_sp
_detector/voice_spoof_detector.py real_voice.wav fake_voice.wav
Similarity Score: 0.9999
Same Speaker (or very close)
PS C:\Users\Ghida\OneDrive\Desktop\DeepfakeDetectionAI>

```

Figure 12: Result showing the real voice was correctly identified

In a subsequent test using the spoofed voice, the system accurately detected it as fake and generated a low similarity score, demonstrating its reliability in identifying deepfake audio.

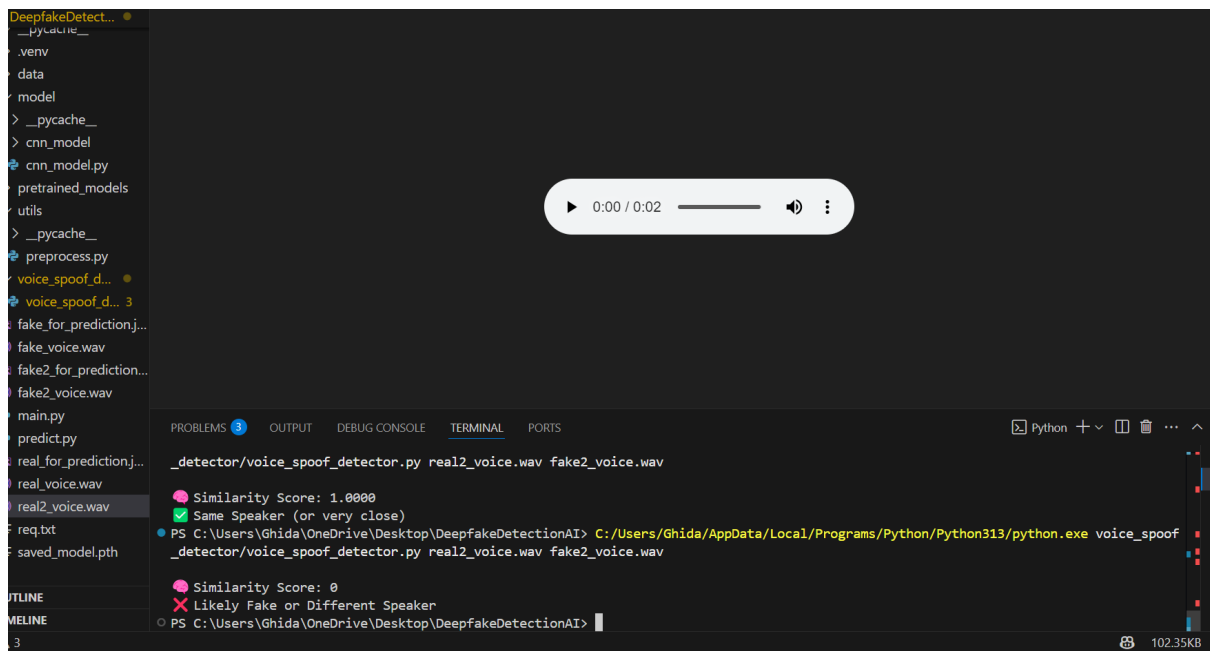


Figure 13: Result showing the fake voice was successfully detected

Graphical User Interface (GUI)

A graphical interface was created to simplify the interaction. The GUI features:

- Username entry
- Upload image for prediction (real/fake)
- Upload voice for prediction (genuine/spoofed)

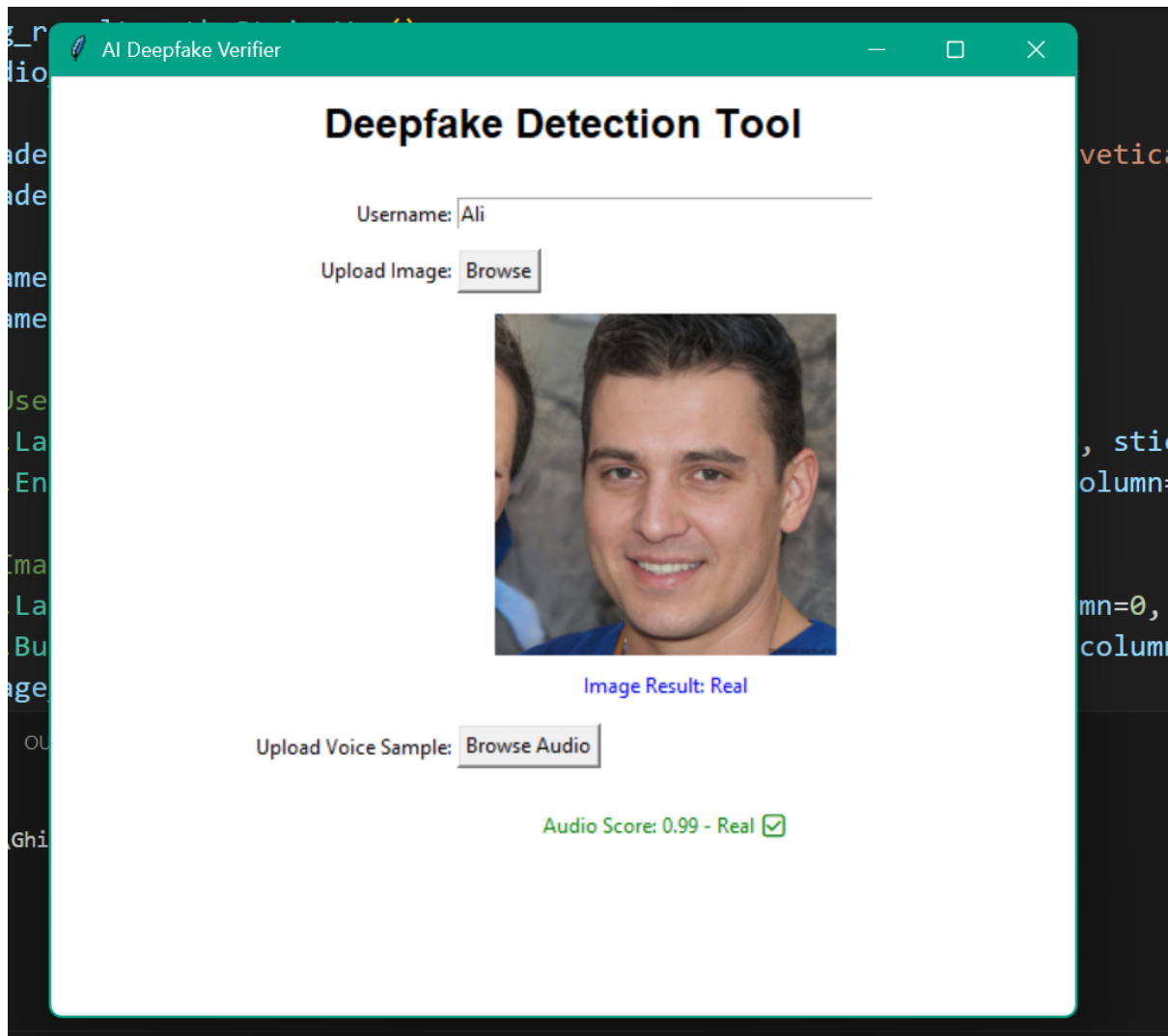


Figure 14: Real image and voice input result

Results shown:

- Image Result: Real
- Voice Result: Same Speaker (Real)
- Similarity Score: 0.99

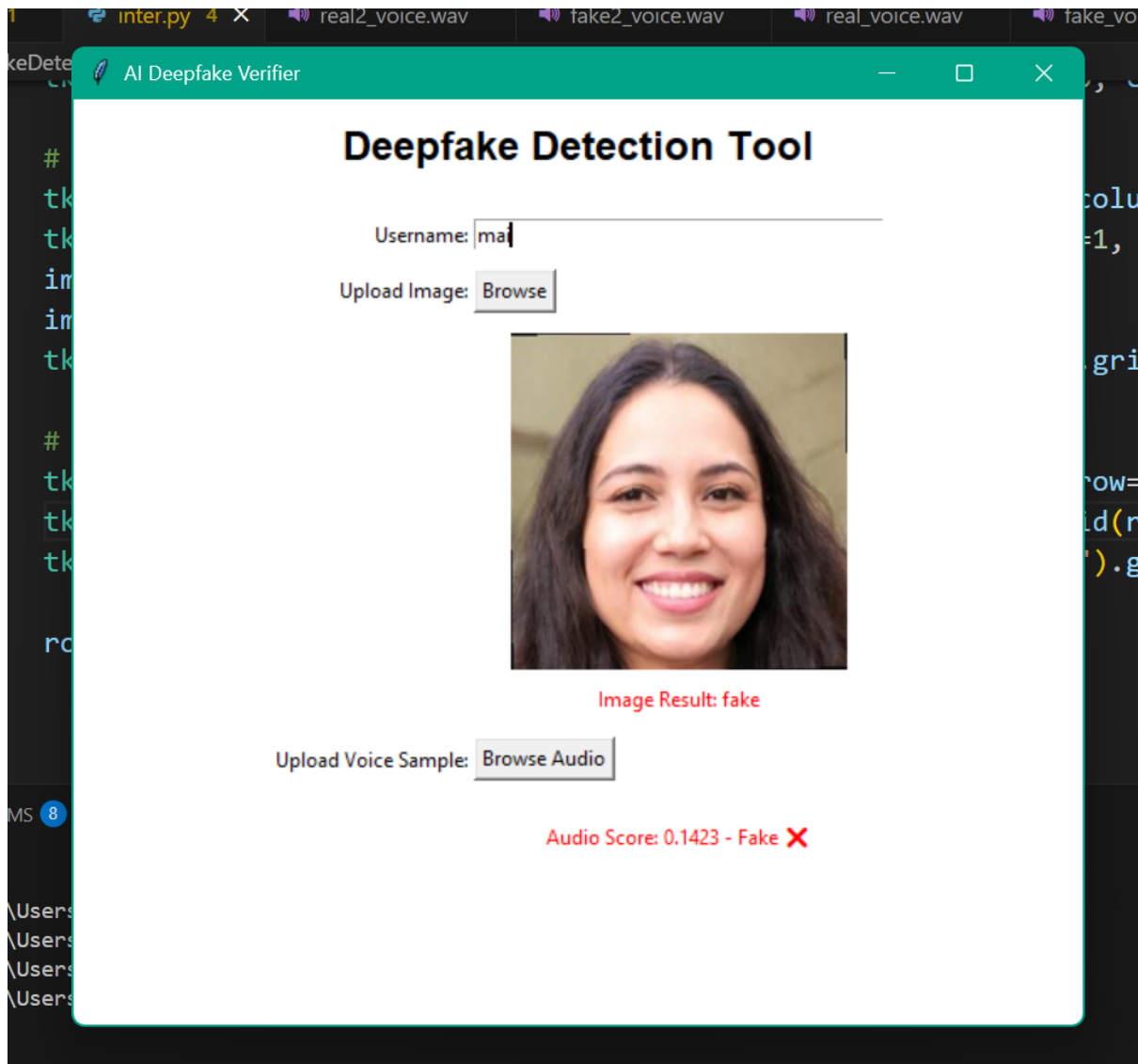


Figure 15: Fake image and voice input result

Results shown:

- Image Result: Fake
- Voice Result: Different Speaker or Spoofed
- Similarity Score: 0.1423

Future Directions

While the current system effectively detects fake images and spoofed voice samples, there are several areas for future enhancement:

- Integrating additional deep learning architectures (e.g., transformers or GAN detectors) to improve detection accuracy.
- Expanding the training dataset with more diverse and real-world examples to enhance generalization.
- Supporting real-time webcam and microphone input for live verification.
- Adding multi-language support and enhancing the user interface design for better accessibility.
- Deploying the system as a web-based or mobile application for broader usage.

Conclusion

This project successfully built a deepfake detection system using both image and audio inputs. Results confirmed that the CNN model and speechbrain pipeline were capable of identifying manipulated content. A GUI was developed to make the system usable for any user, with future improvements including dataset expansion and live recording features.